WITH CLAUSE (CTE)

```sql
CREATE TABLE emp (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept VARCHAR(20),
    age INT,
    salary INT
);
INSERT INTO emp
VALUES
(1,'Ram','HR',25,35000),
(2,'Shyam','IT',32,60000),
(3,'Mansi','Sales',29,55000),
(4,'Rohit','IT',40,85000),
(5,'Priya','HR',35,45000),
(6,'Amit','Sales',28,50000),
(7,'Neha','IT',31,70000),
(8,'Vikas','Finance',45,95000),
(9,'Sneha','Finance',27,48000),
(10,'Karan','Sales',38,65000);
```

emp_id	emp_name	dept	age	salary
1	Ram	HR	25	35000
2	Shyam	IT	32	60000
3	Mansi	Sales	29	55000
4	Rohit	IT	40	85000
5	Priya	HR	35	45000
6	Amit	Sales	28	50000
7	Neha	IT	31	70000
8	Vikas	Finance	45	95000
9	Sneha	Finance	27	48000
10	Karan	Sales	38	65000
NULL	NULL	NULL	NULL	NULL

What is a CTE?

- A CTE (Common Table Expression) is a temporary named result set created using the WITH clause.
- It helps break complex queries into smaller and more readable parts.
- A CTE exists only during the execution of the query.
- It can be referenced like a table within the same query.

```sql
-- Create a CTE named high_salary that contains employees with salary greater than 60000.
-- Display all records from the CTE.
with sal_greater_than_60000  as
    (select * from emp where salary > 60000)
select * from sal_greater_than_60000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept	age	salary
2	Shyam	IT	32	60000
4	Rohit	IT	40	85000
5	Priya	HR	35	45000
7	Neha	IT	31	70000
8	Vikas	Finance	45	95000
10	Karan	Sales	38	65000

```sql
-- Create a CTE named older_employees that contains employees whose age is greater than 30.
-- Display only emp_name and salary.
with older_emp_30 as
    (select * from emp where age > 30)
select emp_name, salary from older_emp_30;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	salary
Shyam	60000
Rohit	85000
Priya	45000
Neha	70000
Vikas	95000
Karan	65000

```sql
-- Create a CTE named it_employees that contains only IT department employees.
-- Display employees whose salary is greater than 65000 from that CTE.
with it_employees as
    (select * from emp where dept = 'IT')
select * from it_employees where salary > 65000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept	age	salary
4	Rohit	IT	40	85000
7	Neha	IT	31	70000

```sql
-- Create a CTE named finance_emp that contains Finance department employees.
-- Display the employee with salary greater than 50000.
with finance_emp as
      (select * from emp where dept = 'finance')
select * from finance_emp  where salary > 50000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept	age	salary
8	Vikas	Finance	45	95000

```sql
-- Create a CTE named experienced_emp that contains employees aged 35 or above.
-- From that CTE, display only employees whose salary is greater than 60000.
with experienced_emp as
      (select * from emp where age > 35)
select * from experienced_emp where salary > 60000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept	age	salary
4	Rohit	IT	40	85000
8	Vikas	Finance	45	95000
10	Karan	Sales	38	65000

```sql
-- Create a CTE named sales_emp that contains Sales department employees.
-- Find the average salary of Sales employees from the CTE.
with sales_emp as
     (select * from emp where dept = 'sales')
select dept, round(avg(salary),2) as avg_salary_sales_emp from sales_emp group by dept;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept	avg_salary_sales_emp
Sales	56666.67

```sql
-- Create a CTE named top_paid containing employees whose salary is greater than the overall average salary.
-- Display all records.
with top_paid_employees as
      (select * from emp where salary >
      (select avg(salary) as average_sal from emp))
select * from top_paid_employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept	age	salary
4	Rohit	IT	40	85000
7	Neha	IT	31	70000
8	Vikas	Finance	45	95000
10	Karan	Sales	38	65000

```sql
-- Create a CTE named it_high_salary containing IT employees with salary greater than 60000.
-- From that CTE display:
/*employee name
department
salary */
-- sorted by salary descending.

with high_salary as
      (select * from emp where dept = 'IT' and salary > 60000)
select emp_name, dept, salary from high_salary order by salary desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	dept	salary
Rohit	IT	85000
Neha	IT	70000

Benefits of CTE

- Improves query readability.
- Makes complex SQL easier to understand.
- Reduces the need for deeply nested subqueries.
- Useful for data transformation and analysis tasks.
- Can be used with recursive queries.

Case Statement

```sql
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    department VARCHAR(20),
    age INT,
    experience INT,
    salary INT
);


INSERT INTO employees
VALUES
(1,'Ram','IT',25,2,45000),
(2,'Shyam','IT',32,7,70000),
(3,'Mansi','HR',28,5,55000),
(4,'Rohit','Finance',41,15,95000),
(5,'Priya','HR',35,10,65000),
(6,'Amit','IT',29,4,52000),
(7,'Neha','Finance',45,18,120000),
(8,'Vikas','Sales',31,6,58000),
(9,'Sneha','Sales',26,3,40000),
(10,'Karan','IT',38,12,85000),
(11,'Pooja','HR',24,1,35000),
(12,'Ajay','Finance',33,8,75000);
```

What is a CASE Statement?

- The CASE statement is used to apply conditional logic in SQL.
- It works similarly to IF-ELSE statements in programming languages.
- It returns different values based on specified conditions.
- Commonly used for categorization, labeling, and conditional calculations.

```sql
/* Q1 Create a column salary_grade.
Rules:
salary >= 100000 → A
salary >= 70000 → B
salary >= 50000 → C
otherwise → D */
select *,
case
    when salary >= 100000 then 'A'
    when salary >= 70000 then 'B'
    when salary >= 50000 then 'C'
    else 'D'
end as salary_grade
from employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	department	age	experience	salary	employee_level	retirement_status	bonus_percent	salary_after_bonus	performance_category	salary_grade
1	Ram	IT	25	2	45000	junior	Active	5	47250	Regular	D
2	Shyam	IT	32	7	70000	mid_level	Active	15	80500	Regular	B
3	Mansi	HR	28	5	55000	mid_level	Active	10	60500	Regular	C
4	Rohit	Finance	41	15	95000	senior	Near Retirement	15	109250	Star performer	B
5	Priya	HR	35	10	65000	senior	Active	10	71500	Regular	C
6	Amit	IT	29	4	52000	junior	Active	10	57200	Regular	C
7	Neha	Finance	45	18	120000	senior	Near Retirement	20	144000	Star performer	A
8	Vikas	Sales	31	6	58000	mid_level	Active	10	63800	Regular	C
9	Sneha	Sales	26	3	40000	junior	Active	5	42000	Regular	D
10	Karan	IT	38	12	85000	senior	Active	15	97750	Star performer	B
11	Pooja	HR	24	1	35000	junior	Active	5	36750	Regular	D
12	Ajay	Finance	33	8	75000	mid_level	Active	15	86250	Regular	B

```sql
/* Q3 Create a column retirement_status.
Rules:
age >= 40 → Near Retirement
otherwise → Active */

alter table employees add column retirement_status char(50) not null;
update employees set retirement_status =
case
    when age >= 40 then 'Near Retirement'
    else 'Active'
end;
```

```sql
 /* Q2 Create a column employee_level.
Rules:
experience >= 10 → Senior
experience >= 5 → Mid-Level
otherwise → Junior */

select *,
case
    when experience >= 10 then 'senior'
    when experience >= 5 then 'mid_level'
    else 'junior'
end as employee_level
from employees;

alter table employees add column employee_level varchar(50) not null;
update employees set employee_level =
(case
    when experience >= 10 then 'senior'
    when experience >= 5 then 'mid_level'
    else 'junior'
end);
```

```sql
/* Q4 Create a column bonus_percent.
Rules:
salary >= 100000 → 20
salary >= 70000 → 15
salary >= 50000 → 10
otherwise → 5 */

alter table employees add column bonus_percent int not null;
update employees set bonus_percent =
case
    when salary >= 100000 then 20
    when salary >= 70000 then 15
    when salary >= 50000 then 10
    else 5
end;

alter table employees add column salary_after_bonus int not null;
update employees set  salary_after_bonus = (bonus_percent/100*salary) + salary;
```

```sql
/* Q5
Display:
emp_name | department | performance_category

Rules:
IT employees with salary > 80000 → Star Performer
Finance employees with salary > 90000 → Star Performer
All others → Regular */

select emp_name, department, salary,
case
    when department = 'IT' and salary > 80000 then 'Star performer'
    when department = 'Finance' and salary > 90000 then 'Star performer'
    else 'Regular'
end as performance_category
from employees;

alter table employees add column performance_category varchar(50) check(performance_category in ('Star performer','Regular'));
update employees set performance_category =
case
    when department = 'IT' and salary > 80000 then 'Star performer'
    when department = 'Finance' and salary > 90000 then 'Star performer'
    else 'Regular'
end;
```

```sql
/* Q6
Create a column salary_band.
Rules:
salary between 30000 and 50000 → Low
salary between 50001 and 80000 → Medium
salary above 80000 → High */

select *,
case
    when salary between 30000 and 50000 then 'low'
    when salary between 50001 and 80000 then 'Medium'
    else 'high'
end as salary_band
from employees;

-- Show department-wise count of employees earning more than 60000.

select department,count(emp_id) from employees where salary > 60000 group by department;

select department,
sum(case
    when salary > 60000 then 1
    else 0
end) as high_salary_count
from employees
group by department;
```

```sql
/* Q7 Display:  department | avg_salary_category
Rules:
average salary > 80000 → High Paying Dept
average salary > 50000 → Medium Paying Dept
otherwise → Low Paying Dept */

-- Using cte
with dept_wise_avg as
    (select department, round(avg(salary),2) as dept_wise_avg from employees group by department)
select department,
case
    when dept_wise_avg > 80000 then 'High Paying Department'
    when dept_wise_avg > 50000 then 'Medium Paying Department'
    else 'Low Paying Department'
end as avg_salary_category
from dept_wise_avg;

-- using subquery
select x1.department,
case
    when dept_wise_avg > 80000 then 'High Paying Department'
    when dept_wise_avg > 50000 then 'Medium Paying Department'
    else 'Low Paying Department'
end as avg_salary_category
     from
          (select department, round(avg(salary),2) as dept_wise_avg from employees group by department) x1;
```

```sql
/* Q8 Display: emp_name | salary | bonus_amount
Rules:
salary >= 100000 → 20% bonus | salary >= 70000 → 15% bonus | salary >= 50000 → 10% bonus | otherwise → 5% bonus */

 -- SUBQUERY
select x1.emp_name, x1.salary, round((x1.bonus_percentage/100)*x1.salary) as bonus_amount from
(select *,
case
    when salary >= 100000 then 20
    when salary >= 70000 then  15
    when salary >= 50000 then 10
    else 5
end as bonus_percentage
from employees)x1;

-- CTE
with tab1 as
(select *,
case
    when salary >= 100000 then 20
    when salary >= 70000 then  15
    when salary >= 50000 then 10
    else 5
end as bonus_percentage
from employees)
select emp_name, salary, round((bonus_percentage/100)*salary) as bonus_amount from tab1;
```

Benefits of CASE

- Simplifies conditional logic in queries.
- Creates custom categories and classifications.
- Useful for reporting and dashboard development.
- Can be combined with aggregate functions such as SUM(), COUNT(), and AVG().

Key Learnings

- Created and used Common Table Expressions (CTEs).
- Compared CTEs with subqueries.
- Implemented conditional logic using CASE statements.
- Built employee categorization and salary grading logic.
- Used CASE with aggregate functions for conditional aggregation.
- Improved query readability and maintainability.
- Practiced real-world SQL scenarios commonly used in data analytics.